

COMP 1324 Algorithmics

Minimum Spanning Trees

Hikmat Farhat

Minimum Spanning Trees

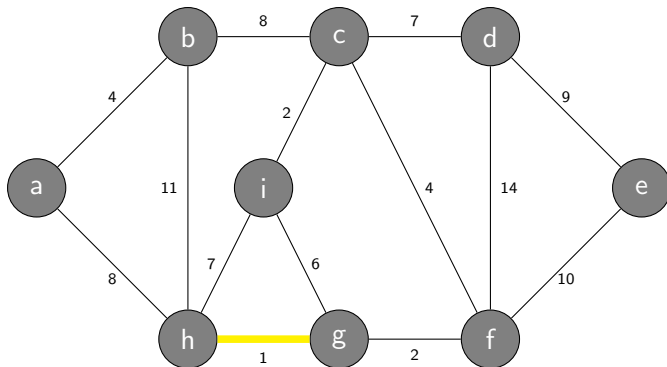
- In many application, when the system is represented by a graph we need to find a **Minimum Spanning Tree** (MST).
- typically we are given a graph $G = (V, E)$ with a weight function $w : E \rightarrow \mathbb{R}$.
- We are looking for a tree $T \subseteq E$ that spans G and has the **least weight**. (weight of a tree is the sum of the weights of its edges).
- As the name suggest this collection of edges T is
 - 1 **A tree**.
 - 2 **Spanning**. meaning connects all the nodes of the graph.
 - 3 It has the **least** weight of all spanning trees.

Kruskal's Algorithm

- Kruskal's algorithm computes a MST of a given graph.
- Every edge has an associated weight or cost.
- The idea is to build the MST by adding an edge every iteration.
 - 1 The edges are considered by increasing order of weight.
 - 2 An edge is added if it doesn't create a cycle.
- The algorithm stops when there are no more edges to consider.

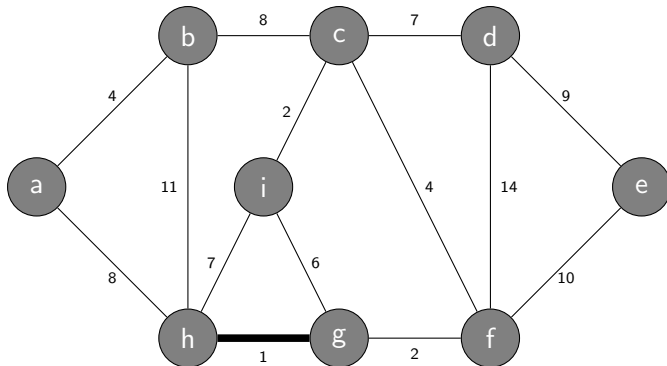
Example (Kruskal's Algorithm)

creates cycle?



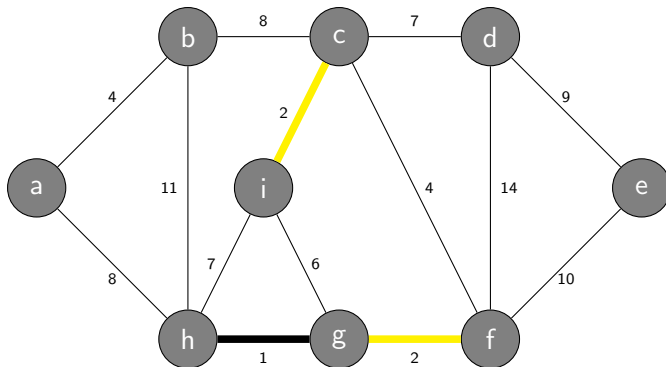
Example (Kruskal's Algorithm)

creates cycle? No, so add it to the MST



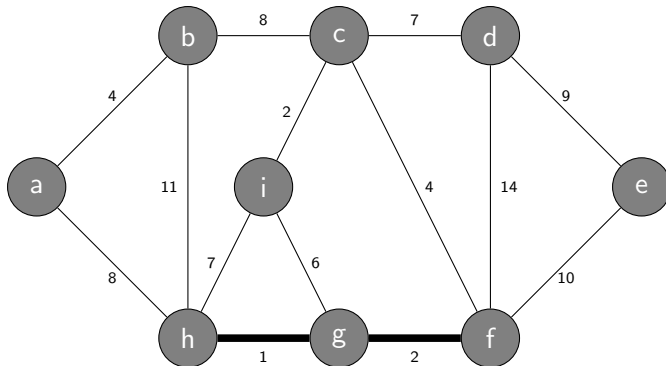
Example (Kruskal's Algorithm)

Two choices



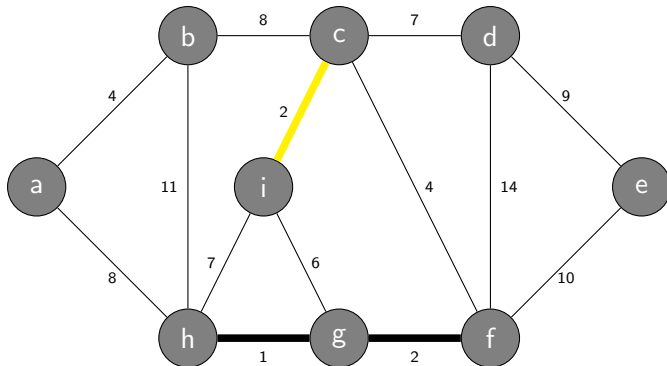
Example (Kruskal's Algorithm)

Two choices. Select one randomly



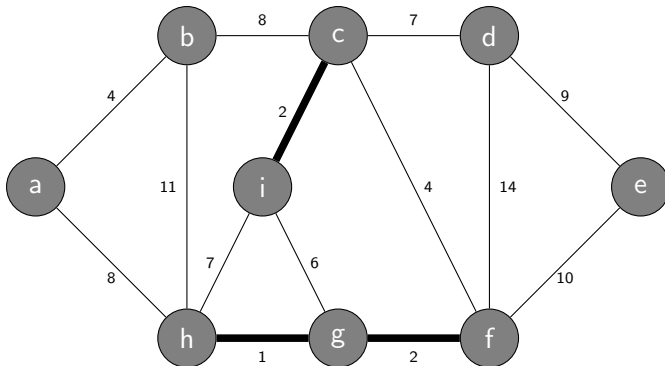
Example (Kruskal's Algorithm)

Creates cycle?



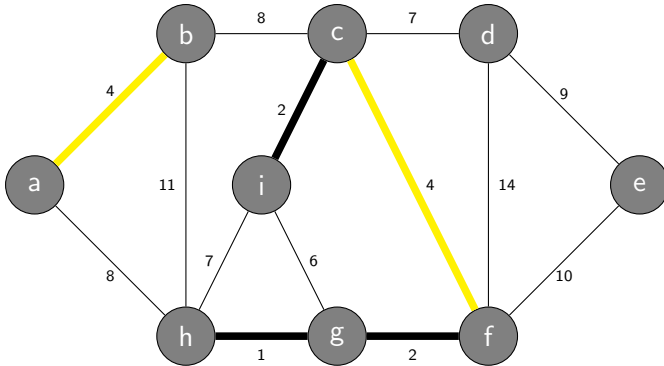
Example (Kruskal's Algorithm)

Creates cycle? No, so add it to the MST



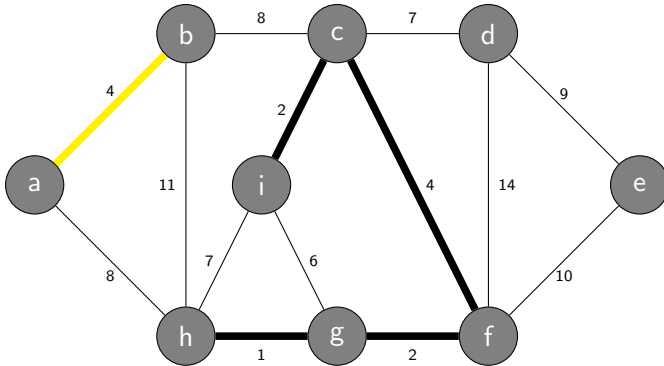
Example (Kruskal's Algorithm)

Two choices.



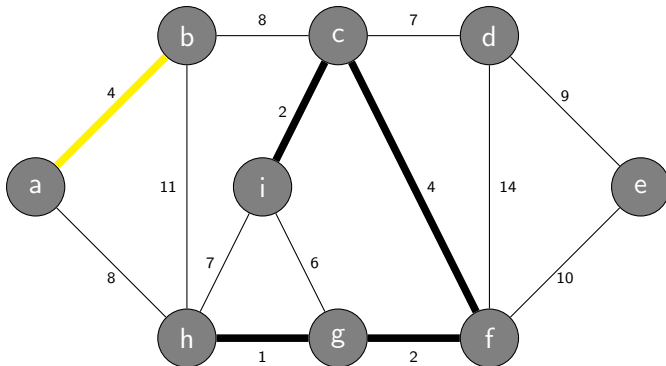
Example (Kruskal's Algorithm)

Two choices. Select one randomly



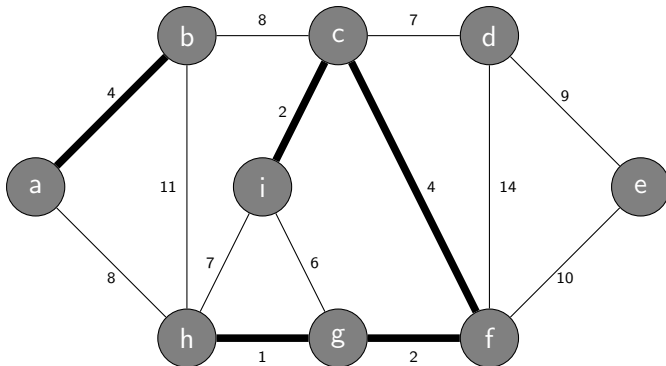
Example (Kruskal's Algorithm)

Creates cycle?



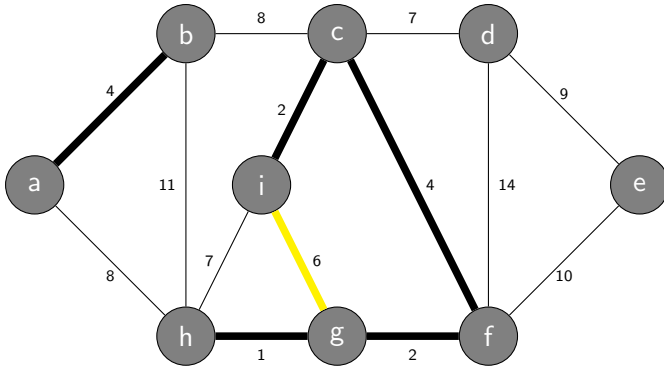
Example (Kruskal's Algorithm)

Creates cycle? No, so add it to the MST



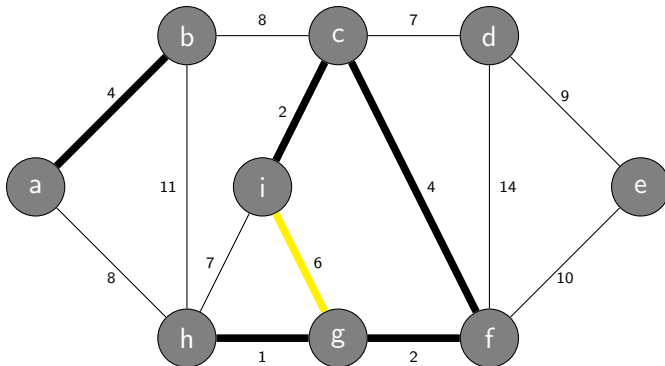
Example (Kruskal's Algorithm)

Creates cycle?



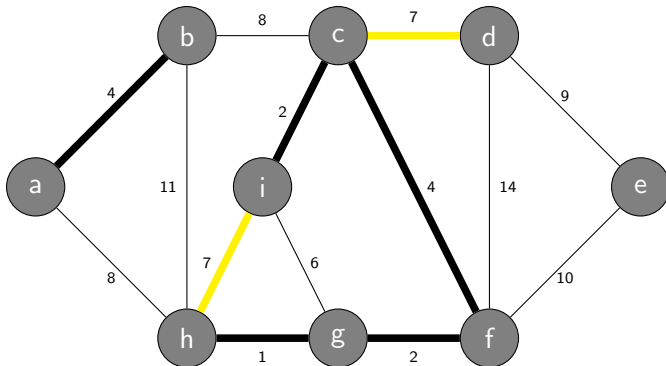
Example (Kruskal's Algorithm)

Creates cycle? Yes, skip this edge



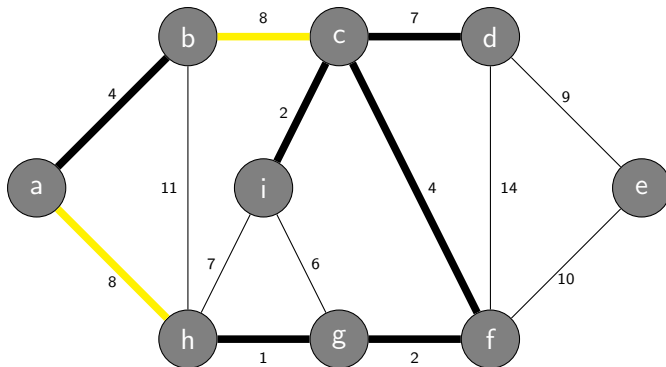
Example (Kruskal's Algorithm)

Two possibilities but one creates a cycle



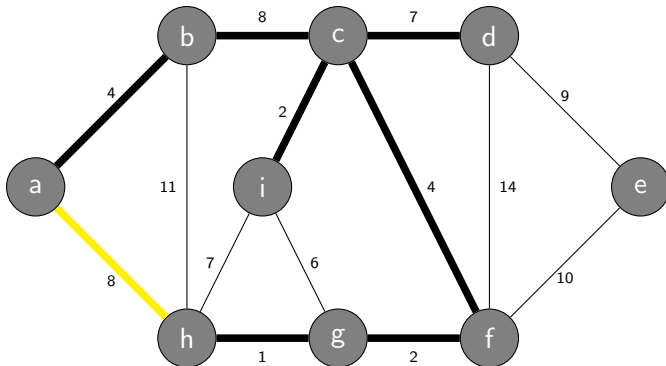
Example (Kruskal's Algorithm)

Two possibilities. Select one randomly.



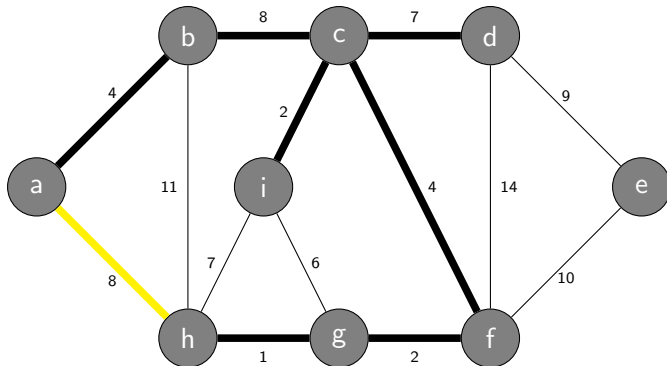
Example (Kruskal's Algorithm)

Creates cycle?



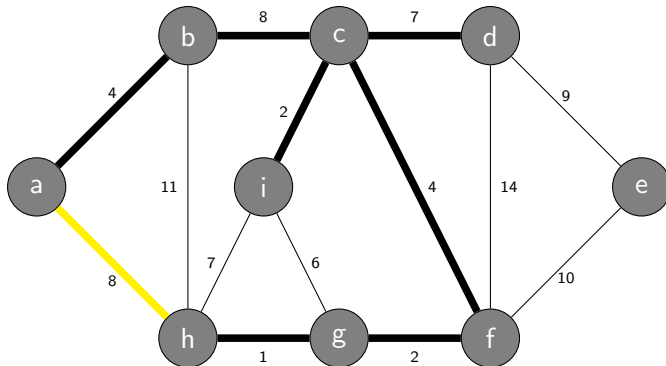
Example (Kruskal's Algorithm)

Creates cycle? Yes, skip this edge.



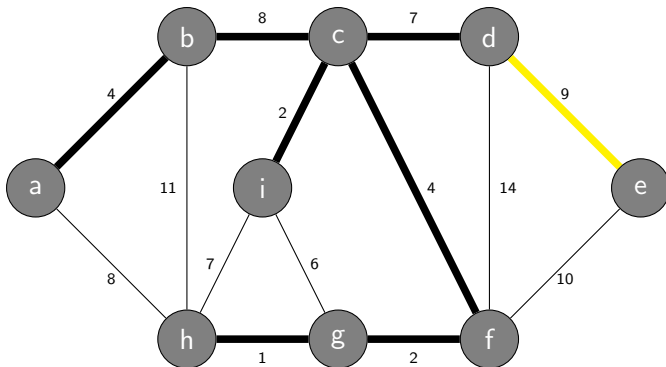
Example (Kruskal's Algorithm)

What if we had selected a-h before b-c?



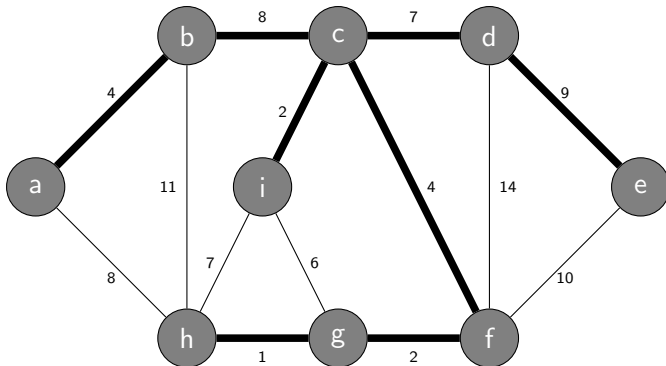
Example (Kruskal's Algorithm)

Creates cycle?



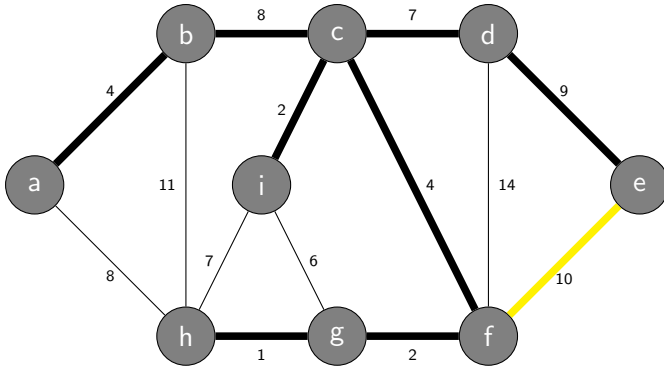
Example (Kruskal's Algorithm)

Creates cycle? No, so add it to the MST.



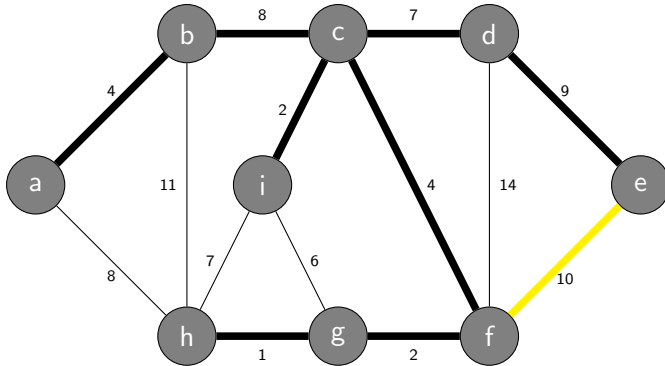
Example (Kruskal's Algorithm)

Creates cycle?



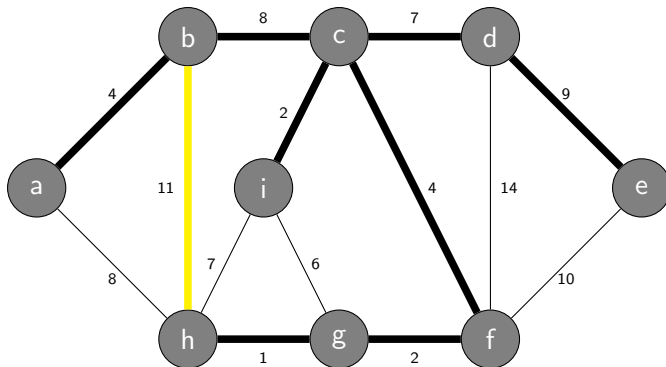
Example (Kruskal's Algorithm)

Creates cycle? Yes so skip this edge.



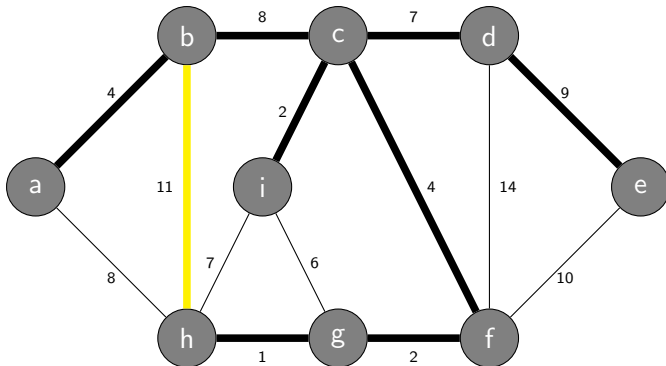
Example (Kruskal's Algorithm)

Creates cycle?



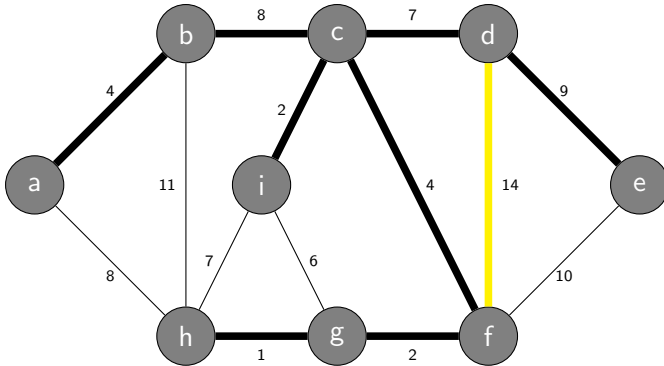
Example (Kruskal's Algorithm)

Creates cycle? Yes so skip this edge.



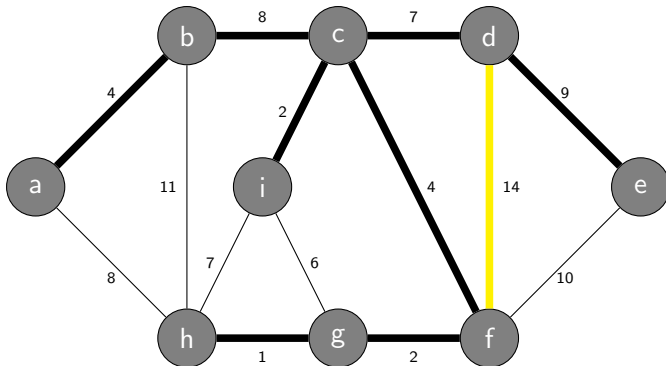
Example (Kruskal's Algorithm)

Creates cycle?



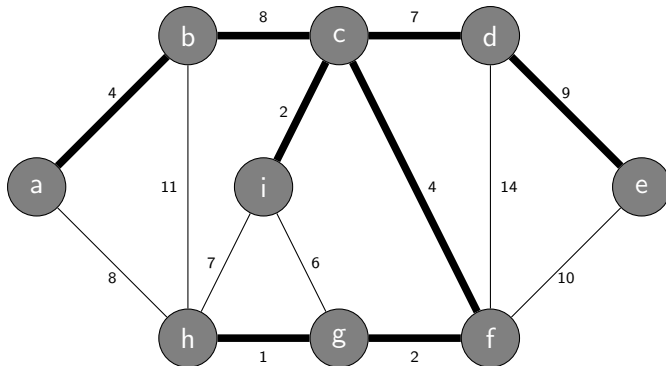
Example (Kruskal's Algorithm)

Creates cycle? Yes so skip this edge.



Example (Kruskal's Algorithm)

No more edges. MST is complete



High level implementation

- Given a graph $G = (V, E)$ with a weight function $w : E \rightarrow \mathbb{R}$.
- We build an MST T by adding edges to T one at a time.
- One can iterate over the edges by increasing weight order by sorting E .
- We need a method to check if adding an edge to the current partial solution creates a cycle.
- This can be done by using the union-find data structure.

Disjoint Sets Data Structures

- Given a graph $G = (V, E)$ and arbitrary vertex $v \in V$.
- $\text{MAKE-SET}(v)$: create a new set whose only member is v .
- $\text{FIND-SET}(v)$: returns a pointer to the representative of the set containing v .
- $\text{UNION}(u, v)$: combine the sets containing u and v into a new set.
- An edge $(u, v) \in E$ creates a cycles iff $\text{FIND-SET}(u) = \text{FIND-SET}(v)$.

Algorithm 1: Kruskal's Algorithm

```
 $T \leftarrow \emptyset$   
foreach  $v \in V$  do  
    | MAKE-SET( $v$ )  
 $F \leftarrow \text{SORT-EDGES}(E)$   
foreach  $(u, v) \in F$  do  
    | if FIND-SET( $u$ )  $\neq$  FIND-SET( $v$ ) then  
        |  $T \leftarrow T \cup \{(u, v)\}$   
        | UNION( $u, v$ )
```

Complexity

Let $n = |V|$ and $m = |E|$.

```

1   $T \leftarrow \emptyset$ 
2  foreach  $v \in V$  do
3       $\text{MAKE-SET}(v)$   $O(1)$ 
4   $F \leftarrow \text{SORT-EDGES}(E)$   $O(m \log m)$ 
5  foreach  $(u, v) \in F$  do  $\Theta(m)$ 
6      if  $\text{FIND-SET}(u) \neq \text{FIND-SET}(v)$  then  $O(\log n)$ 
7           $T \leftarrow T \cup \{(u, v)\}$   $\Theta(1)$ 
8           $\text{UNION}(u, v)$   $O(\log n)$ 
    
```

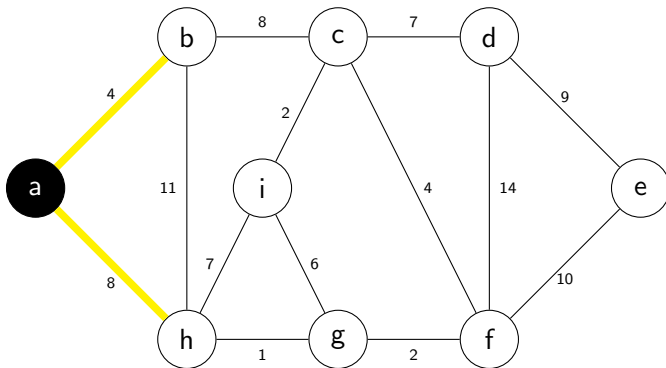
The dominant term is $O(m \log m)$ due to sorting. Often the complexity is written as $O(m \log n)$ since $m \leq n^2$.

Prim's algorithm

- Whereas Kruskal's algorithm is "edge" based, Prim's algorithm is "vertex" based.
- Prim's maintains a set S of vertices, initially containing a single vertex s which could be any vertex in V .
- At each iteration we consider the sets S and $V - S$.
- Find the edge (u, v) with minimum weight such that $u \in S$ and $v \in V - S$.
- Add v to S and (u, v) to T .

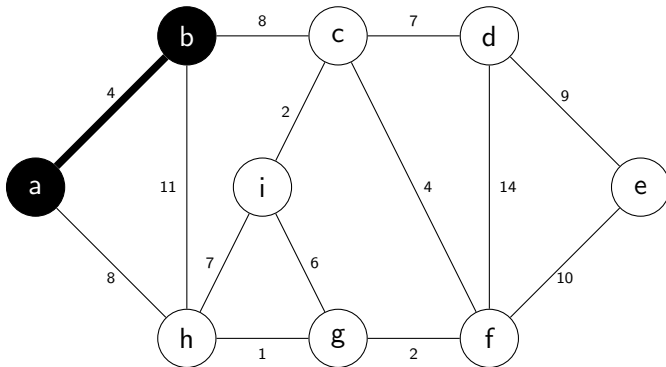
Example (Prim's Algorithm)

$$S = \{a\} \quad V - S = \{b, c, d, e, f, g, h, i\}$$



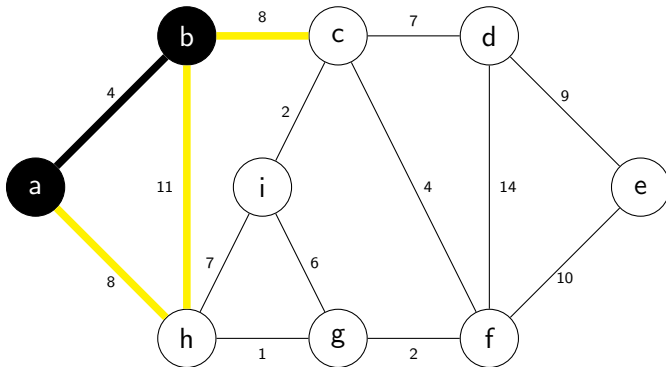
Example (Prim's Algorithm)

$$S = \{a, b\} \quad V - S = \{c, d, e, f, g, h, i\}$$



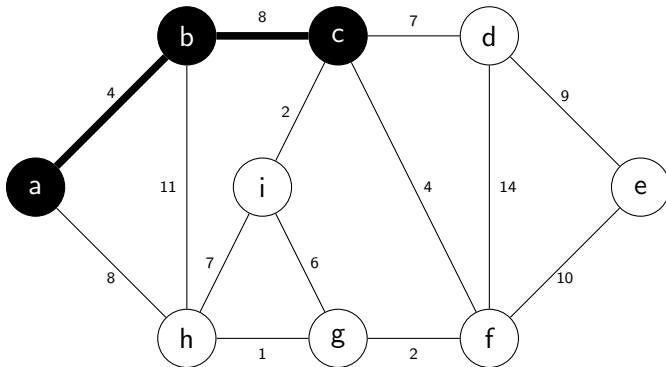
Example (Prim's Algorithm)

$$S = \{a, b\} \quad V - S = \{c, d, e, f, g, h, i\}$$



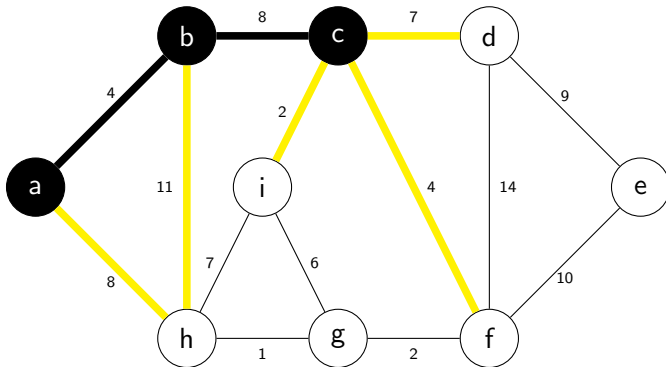
Example (Prim's Algorithm)

$$S = \{a, b, c\} \quad V - S = \{d, e, f, g, h, i\}$$



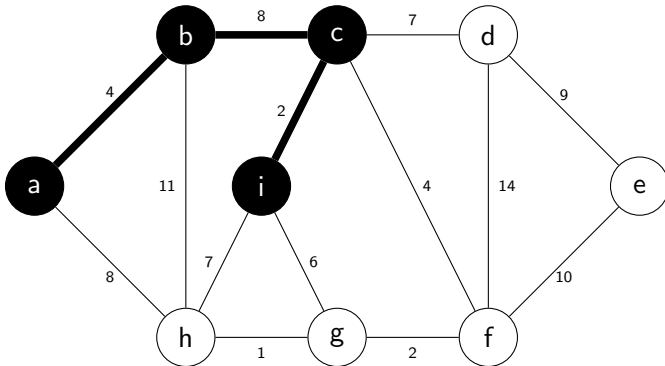
Example (Prim's Algorithm)

$$S = \{a, b, c\} \quad V - S = \{d, e, f, g, h, i\}$$



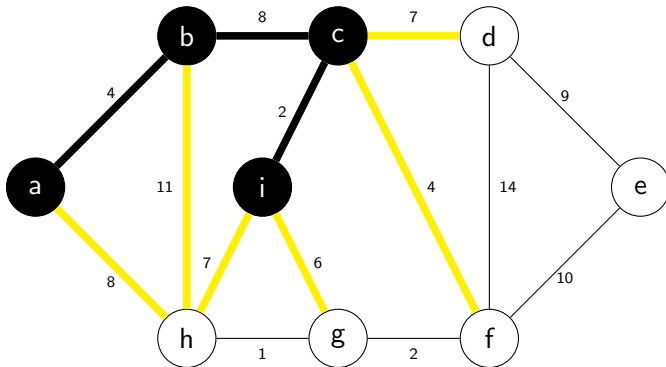
Example (Prim's Algorithm)

$$S = \{a, b, c, i\} \quad V - S = \{d, e, f, g, h\}$$



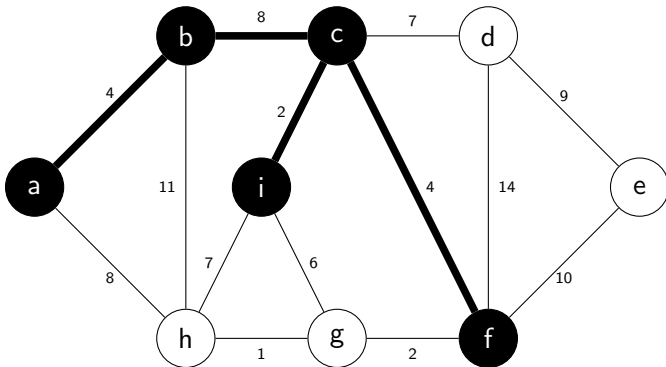
Example (Prim's Algorithm)

$$S = \{a, b, c, i\} \quad V - S = \{d, e, f, g, h\}$$



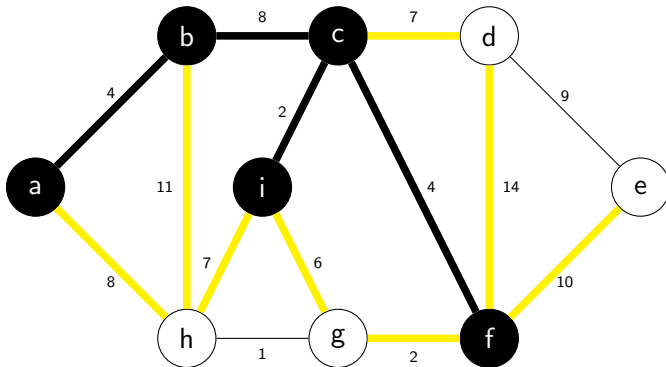
Example (Prim's Algorithm)

$$S = \{a, b, c, i, f\} \quad V - S = \{d, e, g, h\}$$



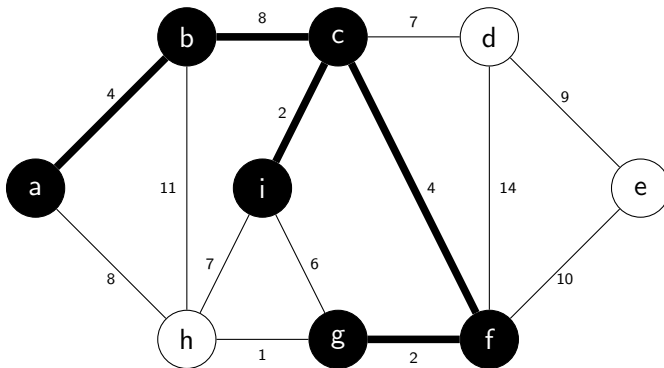
Example (Prim's Algorithm)

$$S = \{a, b, c, i, f\} \quad V - S = \{d, e, g, h\}$$



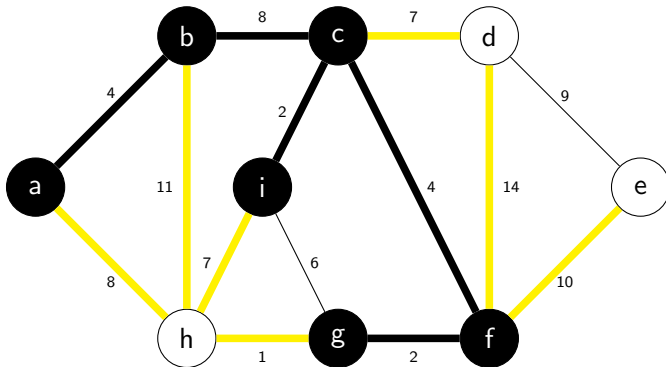
Example (Prim's Algorithm)

$$S = \{a, b, c, i, f, g\} \quad V - S = \{d, e, h\}$$



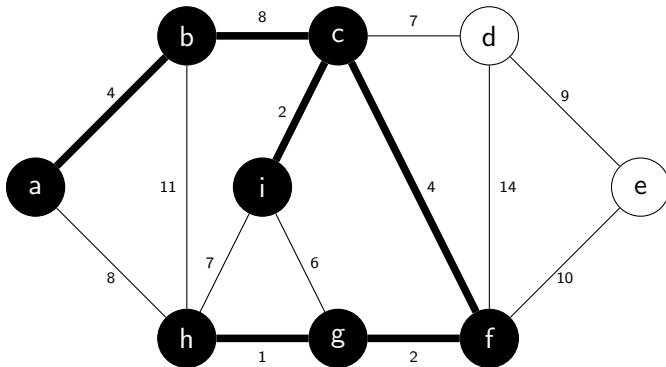
Example (Prim's Algorithm)

$$S = \{a, b, c, i, f, g\} \quad V - S = \{d, e, h\}$$



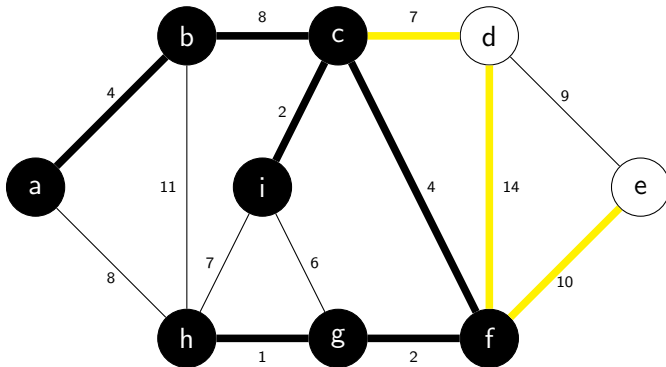
Example (Prim's Algorithm)

$$S = \{a, b, c, i, f, g, h\} \quad V - S = \{d, e\}$$



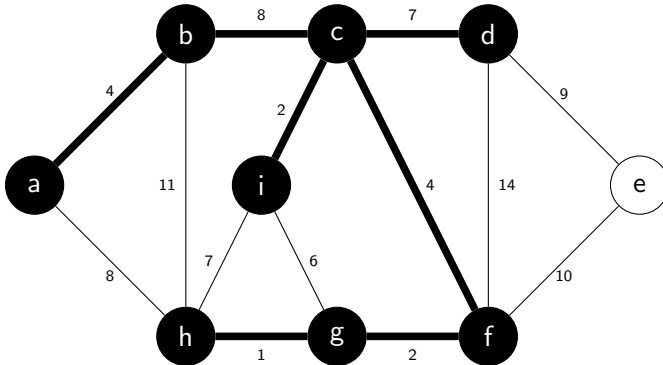
Example (Prim's Algorithm)

$$S = \{a, b, c, i, f, g, h\} \quad V - S = \{d, e\}$$



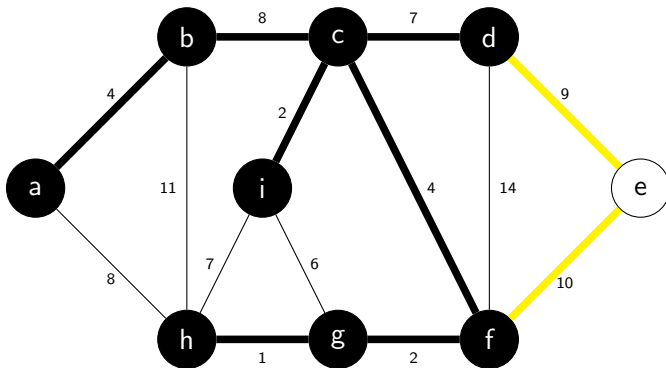
Example (Prim's Algorithm)

$$S = \{a, b, c, i, f, g, h, d\} \quad V - S = \{e\}$$



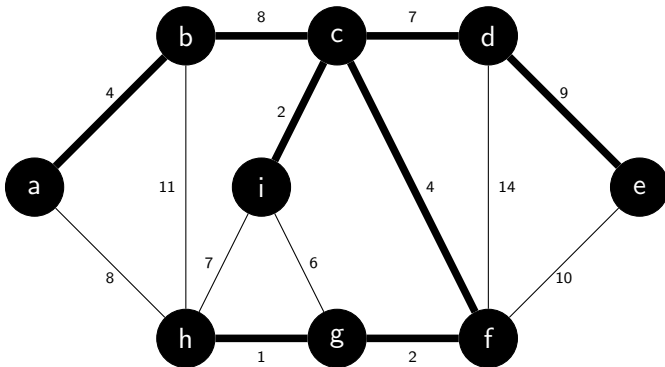
Example (Prim's Algorithm)

$$S = \{a, b, c, i, f, g, h, d\} \quad V - S = \{e\}$$



Example (Prim's Algorithm)

$$S = \{a, b, c, i, f, g, h, d, e\} \quad V - S = \{\}$$



High level implementation

Maintain sets S and $V - S$. Initially $S = \emptyset, T = \emptyset$. Then iterate the following:

- 1 Find $x \in S, y \in V - S$ such that $w(x, y) \leq w(u, v)$ for all $v \in S, u \in V - S$
- 2 Remove y from $V - S$ and add it to S . Also add edge (x, y) to T
- 3 Repeat until $V - S = \emptyset$

The challenge is to **efficiently** implement step 1 because brute force is $O(n^2)$ for each iteration.

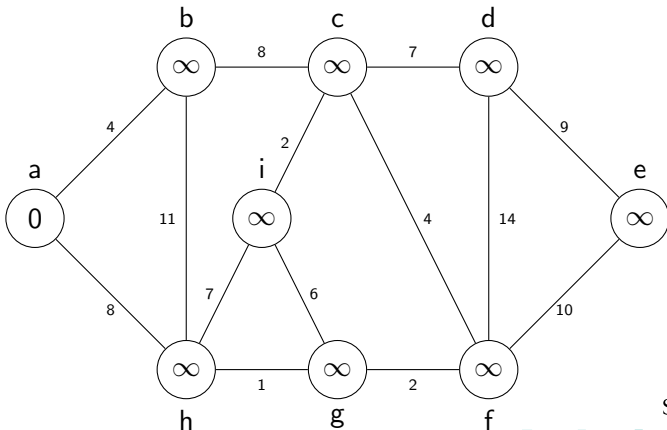
Efficient implementation of step 1

- The trick is to not iterate again over all edges (u, v) , $u \in S, v \in V - S$ to find the one with minimum weight each time, but reuse previous results.
- Suppose that at a given iteration we have computed $w(u, v)$ with $u \in S$ and $v \in V - S$ and saved the result in $v.key$.
- Assume that $w(x, y)$ has the smallest weight among all edges (u, v) with $u \in S$ and $v \in V - S$.
- Then y is removed from $V - S$ and added to S .
- All we have to do is to consider the weights of edges (y, v) for all $v \in adj[y]$.
- Specifically, if $w(y, v) < v.d$ then we update $v.key$ to $w(y, v)$.

Illustrative example

$$S = \{\} \quad V - S = \{a, b, c, d, e, f, g, h, i\}$$

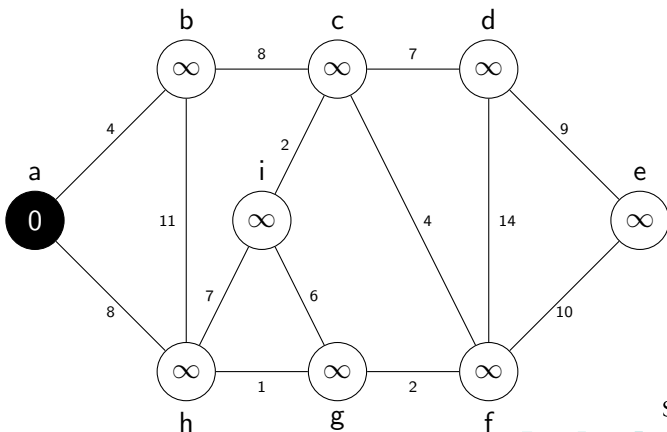
Initially



Illustrative example

$$S = \{a\} \quad V - S = \{b, c, d, e, f, g, h, i\}$$

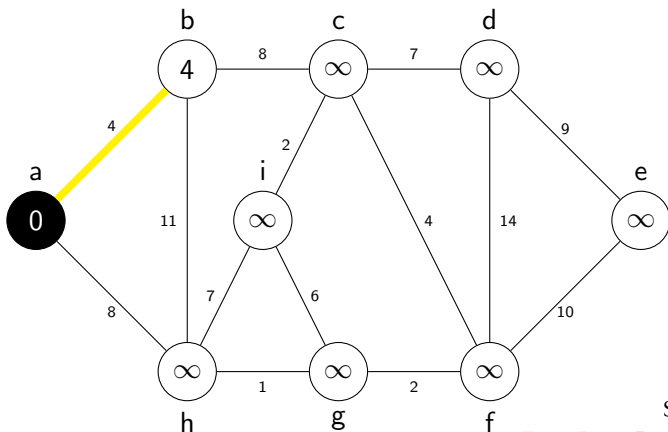
remove vertex with smallest key from $V - S$ and add to S



Illustrative example

$$S = \{a\} \quad V - S = \{b, c, d, e, f, g, h, i\}$$

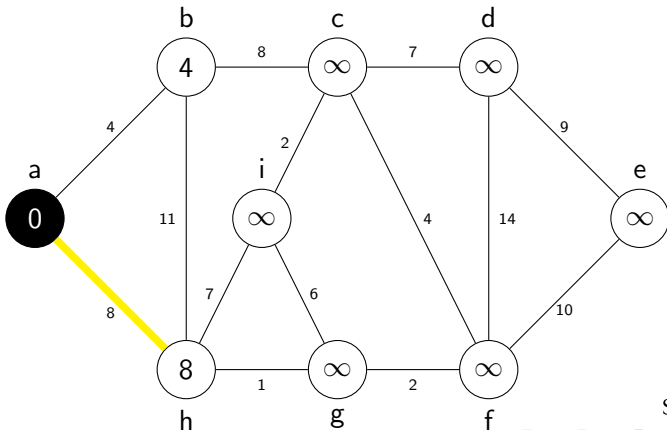
update the neighbors of a



Illustrative example

$$S = \{a\} \quad V - S = \{b, c, d, e, f, g, h, i\}$$

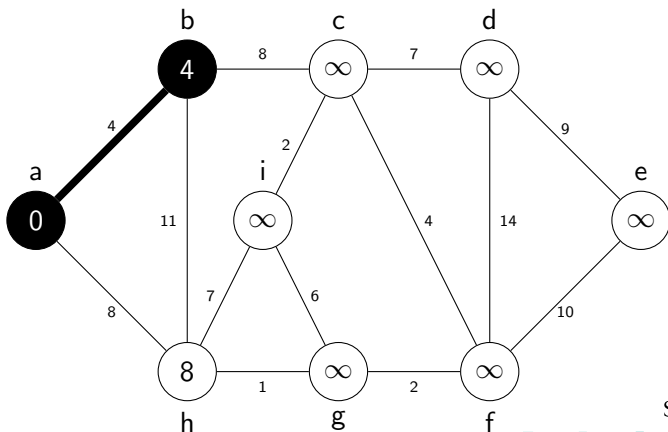
update the neighbors of a



Illustrative example

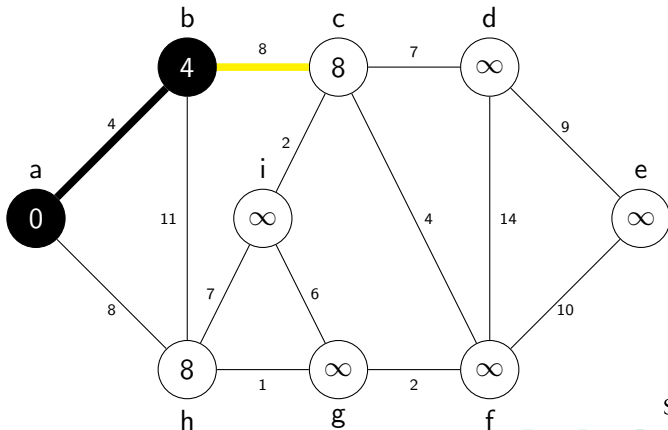
$$S = \{a, b\} \quad V - S = \{c, d, e, f, g, h, i\}$$

remove vertex with smallest key from $V - S$ and add to S



Illustrative example

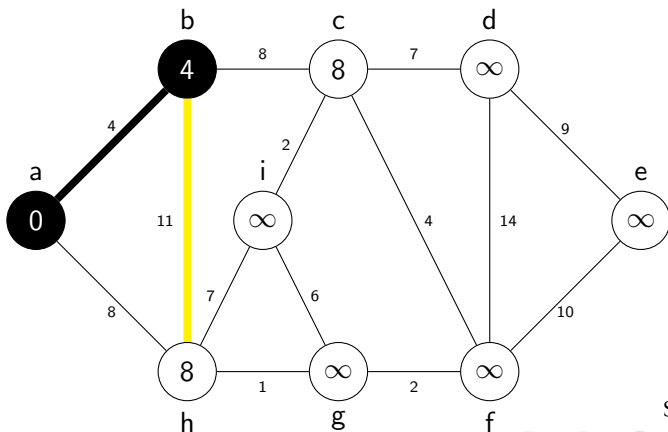
$S = \{a, b\}$ $V - S = \{c, d, e, f, g, h, i\}$
update the neighbors of b



Illustrative example

$$S = \{a, b\} \quad V - S = \{c, d, e, f, g, h, i\}$$

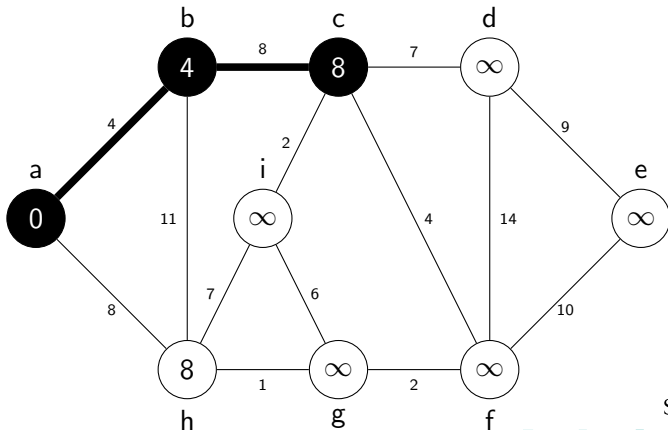
update the neighbors of b



Illustrative example

$$S = \{a, b, c\} \quad V - S = \{d, e, f, g, h, i\}$$

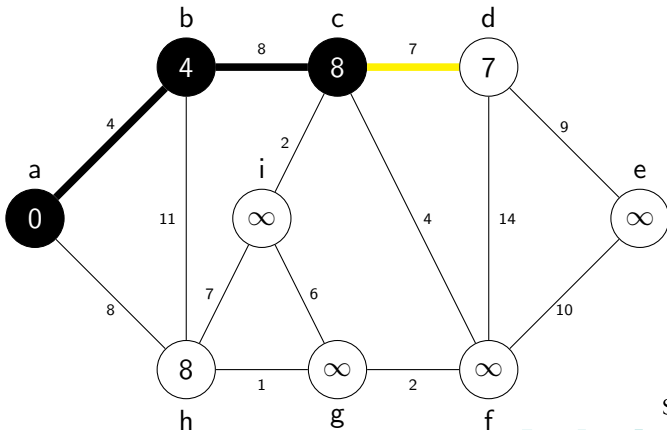
remove vertex with smallest key from $V - S$ and add to S



Illustrative example

$$S = \{a, b, c\} \quad V - S = \{d, e, f, g, h, i\}$$

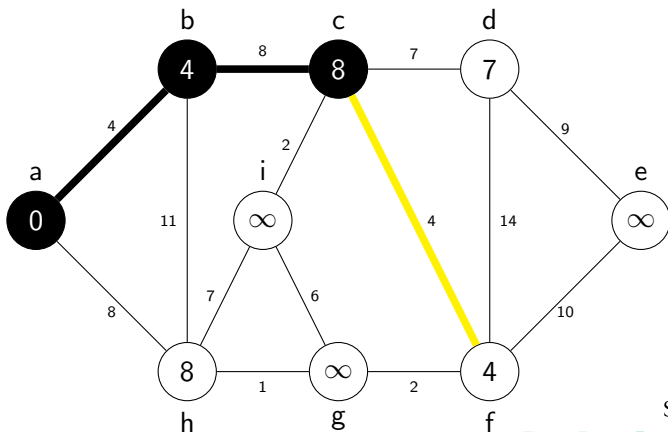
update the neighbors of c



Illustrative example

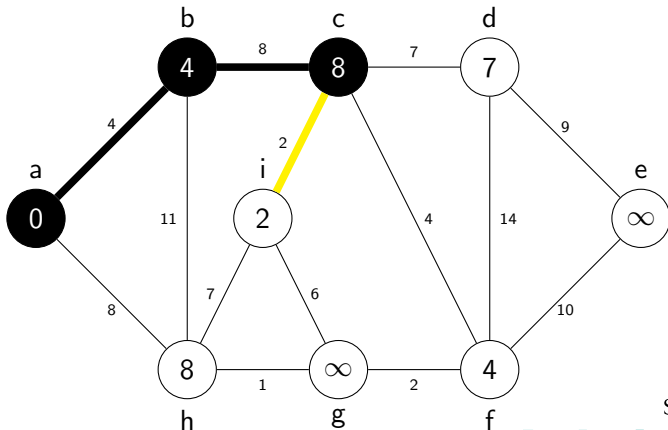
$$S = \{a, b, c\} \quad V - S = \{d, e, f, g, h, i\}$$

update the neighbors of c



Illustrative example

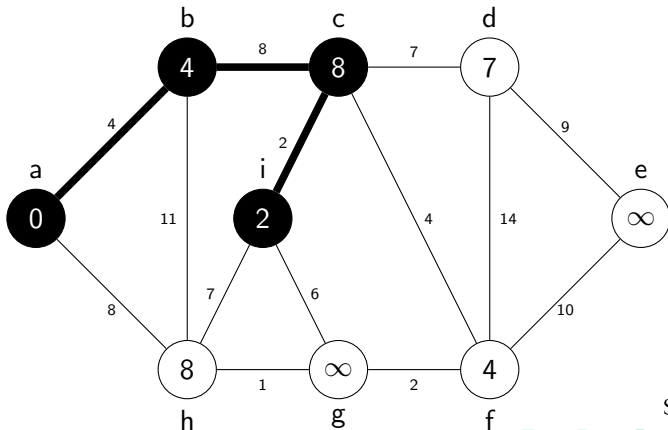
$S = \{a, b, c\}$ $V - S = \{d, e, f, g, h, i\}$
update the neighbors of c



Illustrative example

$$S = \{a, b, c, i\} \quad V - S = \{d, e, f, g, h\}$$

remove vertex with smallest key from $V - S$ and add to S



Pseudo code

Algorithm 2: Prim's algorithm

```

1 foreach  $v \in V$  do
2   |  $v.key, v.p \leftarrow \infty, null$ 
3  $s.key \leftarrow 0$ 
4  $Q \leftarrow V$ 
5  $T \leftarrow \emptyset$ 
6 while  $Q \neq \emptyset$  do
7   |  $u \leftarrow \text{DELETE-MIN}(Q)$ 
8   | if  $u.p \neq null$  then
9     |  $T \leftarrow T \cup \{(u, u.p)\}$ 
10  | foreach  $v \in \text{adj}[u]$  do
11    |   | if  $w(u, v) < v.key$  then
12      |   |   |  $v.key \leftarrow w(u, v)$ 
13      |   |   |  $v.p \leftarrow u$ 
    
```

Notes

- The choice of the "source" vertex on line 4 is arbitrary, any vertex can be chosen as source.
- For clarity an if statement on line 9 was omitted.
- It checks if $u.p$ is null in which case no edge is added to T .
- This occurs only for the first iteration because the source s has no predecessor.
- The usual implementation Q is a priority queue for $\Theta(\log n)$ DELETE-MIN and an extra hash map for $\Theta(1)$ $v \in Q$ operation as well as for the DECREASE-KEY operation.

Complexity

1	foreach $v \in V$ do	$\Theta(n)$
2	$v.key \leftarrow \infty, v.p \leftarrow NULL$	both $\Theta(1)$
3	$s.key \leftarrow 0, T \leftarrow \emptyset$	both $\Theta(1)$
4	$Q \leftarrow V$	$O(n)$
5	while $Q \neq \emptyset$ do	$\Theta(n)$
6	$u \leftarrow \text{DELETE-MIN}(Q)$	$O(\log n)$
7	$T \leftarrow T \cup \{(u, u.p)\}$	$\Theta(1)$
8	foreach $v \in \text{adj}[u]$ do	$ \text{adj}[u] $
9	if $w(u, v) < v.key$ then	$\Theta(1)$
10	$v.key \leftarrow w(u, v)$	$O(\log n)$ Why?
11	$v.p \leftarrow u$	$\Theta(1)$

The dominant term comes from the execution of line 10 which is $O(\log n)$ and executed $O(m)$ times for a total of $O(m \log n)$

General MST algorithm

- Both Kruskal's and Prim's algorithms are special cases of a general method to obtain a minimum spanning tree.
- The basic idea is based on the following:
- Maintain a set of edges A .
- Before every iteration A is a subset of some minimum spanning tree.
- At each step we add an edge (u, v) to A such that $A \cup \{(u, v)\}$ is **still** a subset of some MST.
- An edge having that property is called **safe** for A .

General MST algorithm

Algorithm 3: General MST algorithm

$\text{MST}(G)$

$A \leftarrow \emptyset$

while A is not MST **do**

 find edge (u, v) safe for A

$A \leftarrow A \cup \{(u, v)\}$

return A

Some Definitions

- Let $G = (V, E)$ be a graph with some real-valued weight function $w : E \rightarrow R$.
- A **cut** $(S, V - S)$ of the graph G is a **partition** of V .
- We say a cut $(S, V - S)$ **respects** $A \subseteq E$ if no edge in A crosses the cut.
- An edge is said to be a **light edge** crossing a cut if its weight is the minimum of any edge crossing the cut.

This is why it works

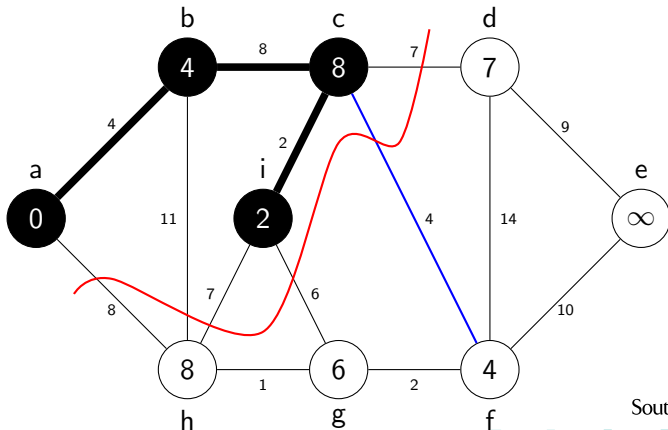
- The reason why both algorithms work is the following theorem

Theorem

Let A be a set of edges included in some minimum spanning tree, $(S, V - S)$ a cut that respects A , and (u, v) be a light edge crossing $(S, V - S)$. Then (u, v) is safe for A .

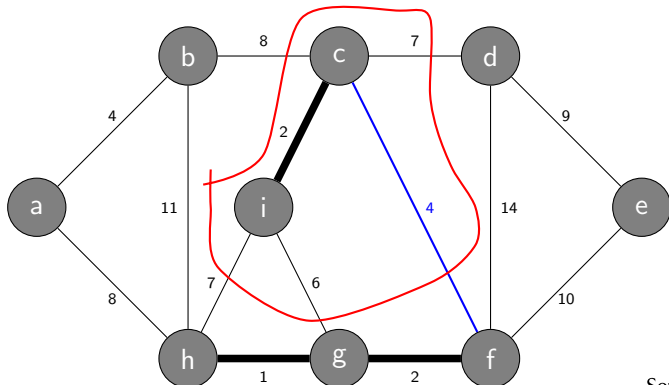
Example

Cut $V - Q = \{a, b, c, i\}$ $Q = \{d, e, f, g, h\}$
Cut respects $A = \{(a, b), (b, c), (c, i)\}$ and (c, f) is a light edge



Example

Cut $S = \{c, i\}$ and $V - S$ respects $A = \{(h, g), (g, f), (c, i)\}$
 (c, f) is a light edge crossing the cut



Correctness of Prim's Algorithm (Optional)

- At the beginning of every iteration (except the first) Prim's algorithm selects a vertex u for removal from Q , where $u.key$ is minimum.
- This means that $(u.p, u)$ is a light edge for the cut $(Q, V - Q)$
- Furthermore, the partially constructed tree T contains edges that connect vertices in $V - Q$ thus the cut respects T
- Therefore $(u.p, u)$ is safe for T .

Correctness of Kruskal's Algorithm (Optional)

- Prior to every iteration of Kruskal's algorithm we have
- A forest (a collection of trees) $G_A = (V, A)$. (initially is A is empty)
- Select an edge $(u, v) \in E - A$ with
 - $w(u, v)$ is minimal.
 - $u \in T_u$ and $v \notin T_u$ where T_u is a tree in G_A that contains u .
- From the above we have that: $(T_u, V - T_u)$ is a cut that respects A and (u, v) is a light edge crossing that cut.
- From the theorem we know that (u, v) is a safe edge for A .

Proof (Optional)

- The figure below shows a cut $(S, V - S)$ where the black vertices are in S and the whites in $V - S$. The edges in A are the shaded edges so the cut respects A .
- Suppose $A \subseteq T$ where T is a minimum spanning tree and (u, v) is a light edge crossing the cut.

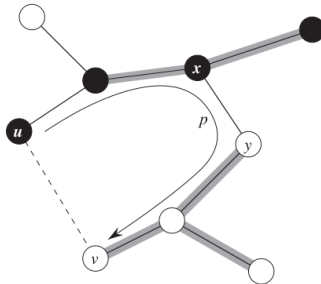


Figure: From CLRS

Proof(Optional)

- Assume $(u, v) \notin T$, otherwise we are done.
- Since T is an MST then $(x, y) \in T$.
- Let $T' = T \cup \{(u, v)\} - (x, y)$.
- $w(T') = w(T) + w(u, v) - w(x, y) \leq w(T)$ so T' is an MST.
- Furthermore,

$$\begin{aligned}
 A \subseteq T \wedge (x, y) \notin A &\Rightarrow A \subseteq T - \{(x, y)\} \\
 &\Rightarrow A \cup \{(u, v)\} \subseteq T - \{(x, y)\} \cup \{(u, v)\} \\
 &\Rightarrow A \cup \{(u, v)\} \subseteq T'
 \end{aligned}$$

- Thus (u, v) is safe for A .